~/majibill/docs/04-schema-erd.md

$ cat 04-schema-erd.md

MAJIBILL PLATFORM · DATA MODEL

Database Schema & ERD

Entity-relationship diagrams, table catalogue, key DDL, constraints, lifecycles, partitioning and retention for the MajiBill data model.

```
CREATE UNIQUE INDEX ON payments (tenant_id, provider, provider_txn_id);
-- idempotency is a constraint, not a hope
-- ledger_entries: INSERT only, balanced per journal
```

DOC	VERSION	DATE
MB-04	1.0 draft	08 Oct 2026

STATUS	AUDIENCE	CLASS
For review	Backend, data and QA engineers	Confidential

Prepared by 404HubSpot · Nakuru, Kenya · https://404-hub-spot.vercel.app
Working codename: MajiBill (Swahili: maji = water). Figures marked 'assumption' must be validated before commitment.

// Contents

1. Conventions . . . 3
2. ERD 1 — Platform and identity . . . 3
3. ERD 2 — Customers, assets and operations . . . 4
4. ERD 3 — Tariffs, readings and billing . . . 5
5. ERD 4 — Ledger and payments . . . 6
6. Lifecycles . . . 8
7. Partitioning, indexing and retention . . . 8

// 1. Conventions

Topic	Convention
Naming	`snake_case`, plural table names, `*_id` for foreign keys, `*_at` for timestamps (timestamptz, UTC), `*_minor` for money in minor units
Keys	UUIDv7 primary keys. Business identifiers (`account_no`, `bill_no`, `receipt_no`) are unique **per tenant** via `(tenant_id, number)`
Tenancy	Every tenant-owned table has `tenant_id uuid NOT NULL`, an RLS policy, and `tenant_id` as the leading column of its indexes
Foreign keys	Composite where it matters: `(tenant_id, customer_id)` references `customers (tenant_id, id)` so a row can never point at another tenant's parent
Money	`bigint` minor units + `currency char(3) DEFAULT 'KES'`; rates `numeric(12,4)`
Status fields	Text with `CHECK` constraints (or enum) and a transition table enforced in the application and by trigger for money tables
Soft delete	Not used for financial data. Customer PII erasure = anonymise fields; rows remain
Timestamps	`created_at`, `updated_at` on mutable tables; append-only tables carry `created_at` only

> **DECISION** **Composite foreign keys enforce tenancy at the schema level**
> RLS stops cross-tenant reads. Composite FKs stop cross-tenant **references**. Together, a bug cannot attach tenant A's bill to tenant B's account, even with a wrong id.

// 2. ERD 1 — Platform and identity

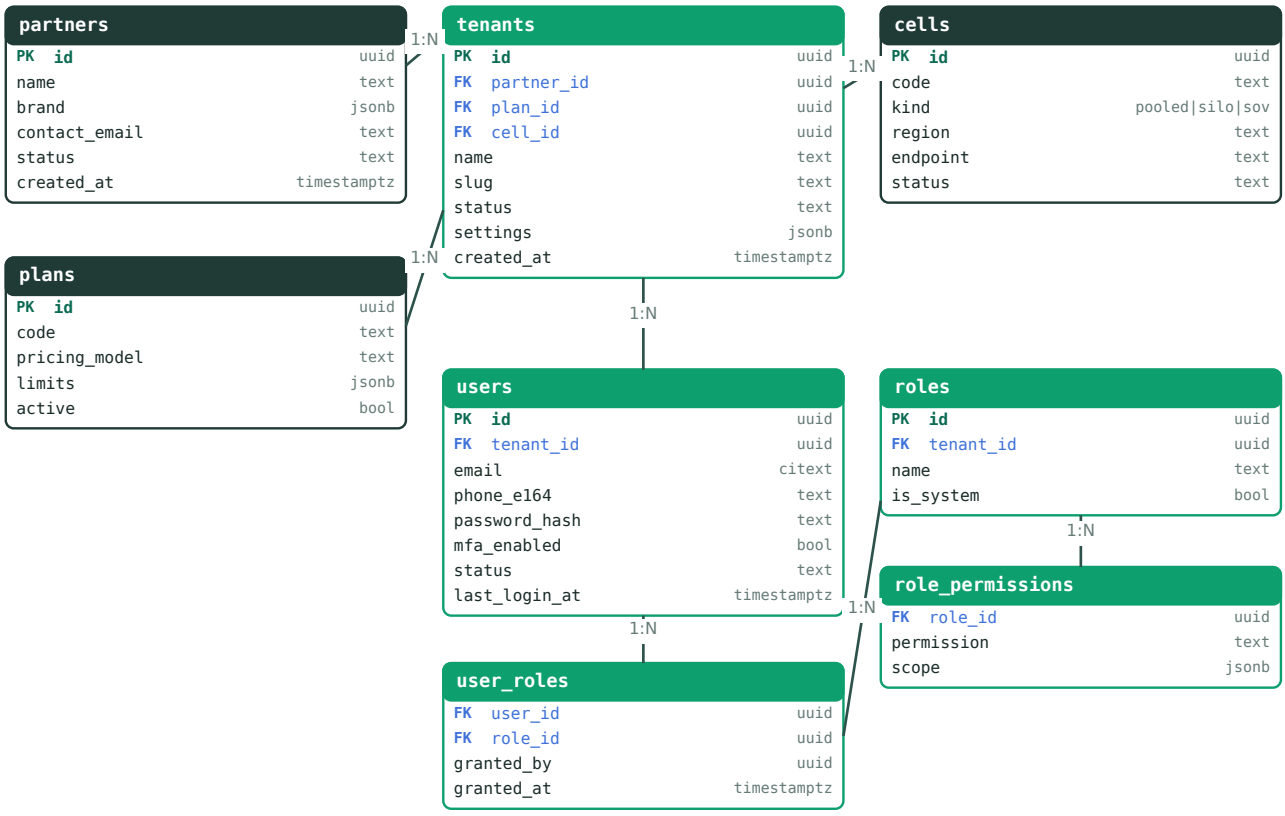


Fig 2.1 Control-plane tables (dark) live in the platform database; tenant-scoped identity tables (green) carry tenant_id + RLS.

`partners`, `plans`, `cells` and `tenants` are control-plane tables: they exist once, outside tenant RLS, owned by the platform role. `users`, `roles` and their children are tenant-scoped. Platform staff authenticate separately (`platform_users`) and reach tenant data only via audited support sessions (MB-06).

// 3. ERD 2 — Customers, assets and operations

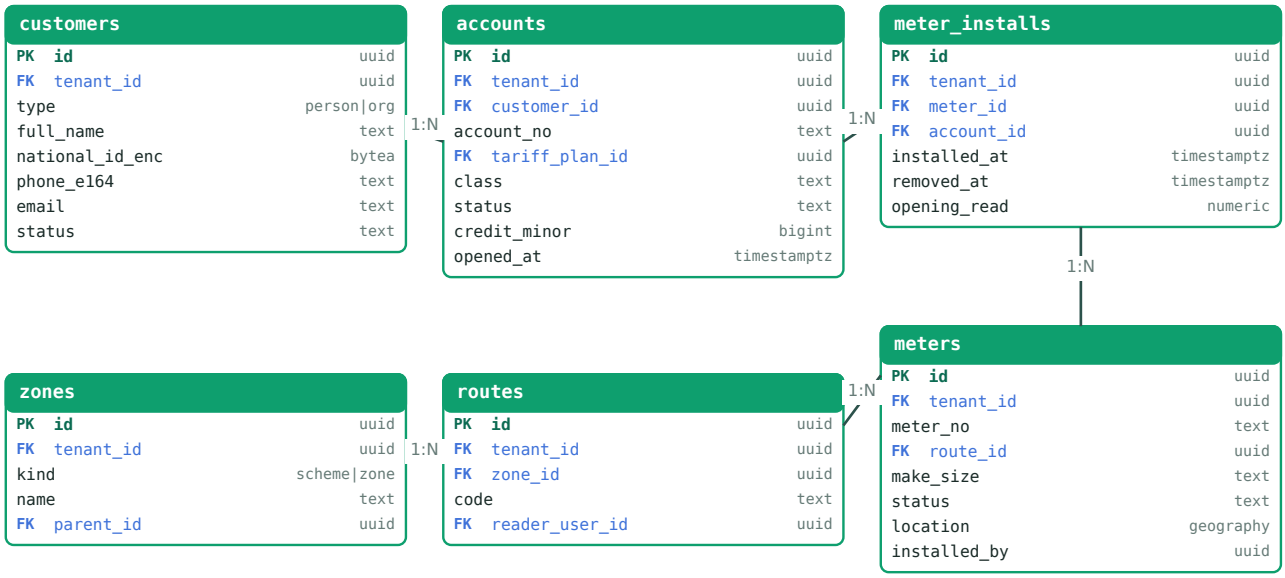


Fig 3.1 A meter can move between accounts over time; meter_installs records the history, and readings attach to the install period.

Table	Key columns / constraints	Notes
customers	UNIQUE (tenant_id, phone_e164) where active; national_id_enc encrypted	PII; field-level encryption; anonymisable
accounts	UNIQUE (tenant_id, account_no); FK (tenant_id, customer_id)	Account number is the M-Pesa reference; includes check digit
meter_installs	Exclusion constraint: one open install per meter (daterange no overlap)	Gives consumption history across replacements
meters	UNIQUE (tenant_id, meter_no)	GPS point optional; photo on install
routes, zones	Zone tree via parent_id	Drive reader assignment and reporting

// 4. ERD 3 — Tariffs, readings and billing

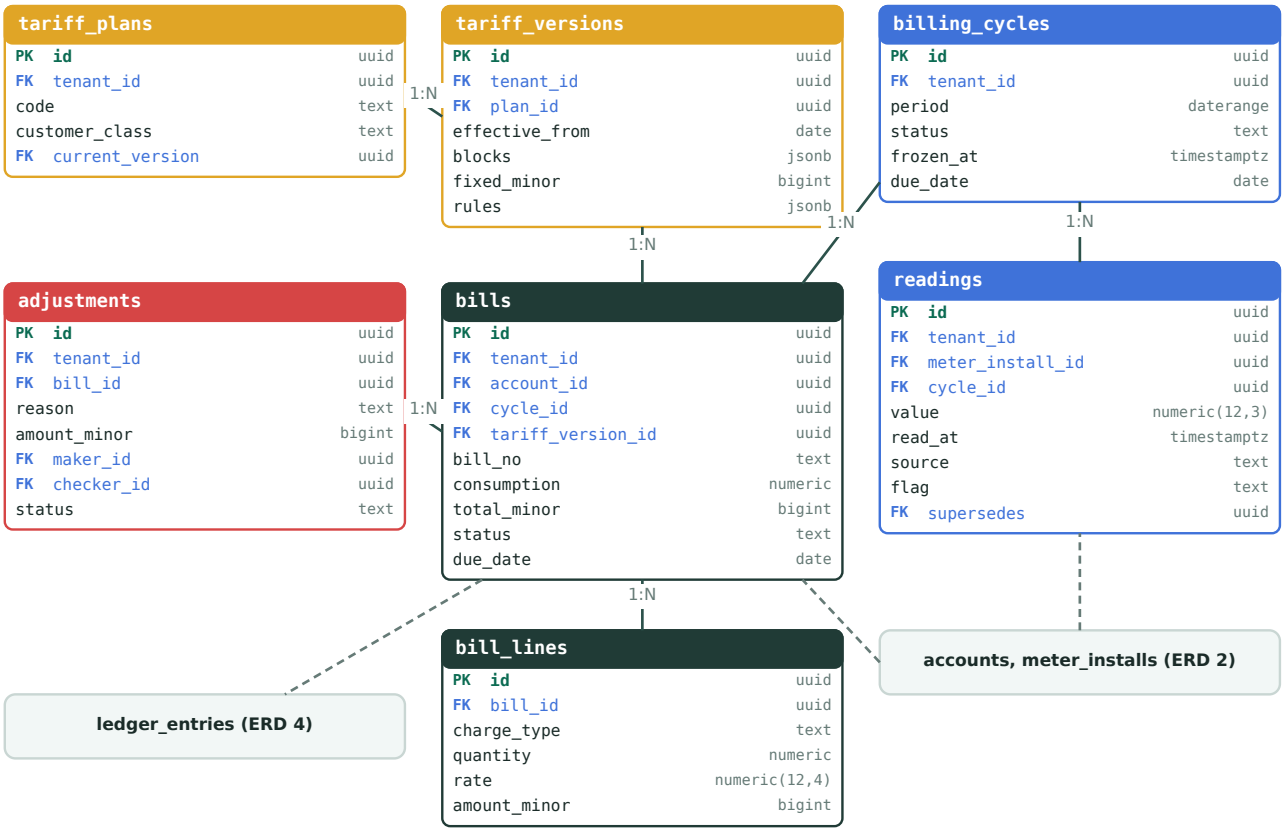


Fig 4.1 Bills reference the tariff version used and the cycle they belong to; lines make each bill fully explainable. Dashed = relation to another diagram.

// 5. ERD 4 — Ledger and payments

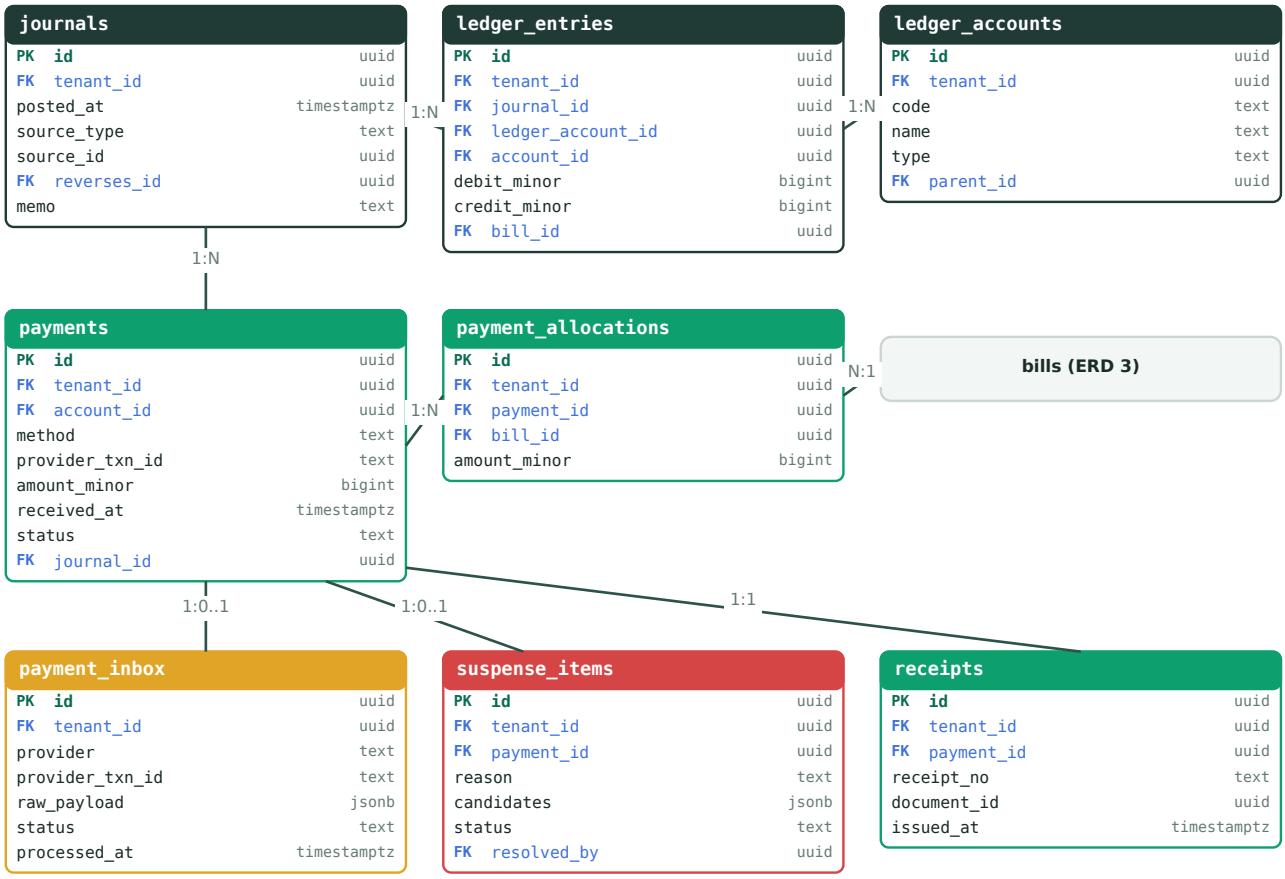


Fig 5.1 Payment inbox → payment → journal/ledger entries, with suspense and receipts as side effects.

5.1 Key DDL (excerpts)

```sql
CREATE TABLE payment_inbox (
  id               uuid PRIMARY KEY DEFAULT uuid_generate_v7(),
  tenant_id        uuid NOT NULL REFERENCES tenants(id),
  provider         text NOT NULL,                      -- 'mpesa_c2b', 'bank_csv', ...
  provider_txn_id  text NOT NULL,
  raw_payload      jsonb NOT NULL,
  status           text NOT NULL DEFAULT 'received'
                   CHECK (status IN ('received','processing','done','failed','suspense')),
  received_at      timestamptz NOT NULL DEFAULT now(),
  processed_at     timestamptz,
  UNIQUE (tenant_id, provider, provider_txn_id)        -- idempotency is a constraint
);
```

```sql
```

```
CREATE TABLE ledger_entries (
  id                uuid NOT NULL DEFAULT uuid_generate_v7(),
  tenant_id         uuid NOT NULL,
  journal_id        uuid NOT NULL,
  ledger_account_id  uuid NOT NULL,
  account_id        uuid,                     -- customer account when relevant
  debit_minor       bigint NOT NULL DEFAULT 0 CHECK (debit_minor  >= 0),
  credit_minor      bigint NOT NULL DEFAULT 0 CHECK (credit_minor >= 0),
  currency          char(3) NOT NULL DEFAULT 'KES',
  bill_id           uuid,
  created_at        timestamptz NOT NULL DEFAULT now(),
  CHECK ((debit_minor = 0) <> (credit_minor = 0)),     -- exactly one side
  PRIMARY KEY (tenant_id, created_at, id)
) PARTITION BY RANGE (created_at);
REVOKE UPDATE, DELETE, TRUNCATE ON ledger_entries FROM app_role;
-- deferred constraint trigger asserts SUM(debit) = SUM(credit) per journal at commit
```

● ● ● sql

```
CREATE TABLE meter_installs (
  id uuid PRIMARY KEY DEFAULT uuid_generate_v7(),
  tenant_id uuid NOT NULL, meter_id uuid NOT NULL, account_id uuid NOT NULL,
  installed_at timestamptz NOT NULL, removed_at timestamptz,
  EXCLUDE USING gist (tenant_id WITH =, meter_id WITH =,
                      tstzrange(installed_at, removed_at) WITH &&)   -- no overlapping installs
);
```

// 6. Lifecycles

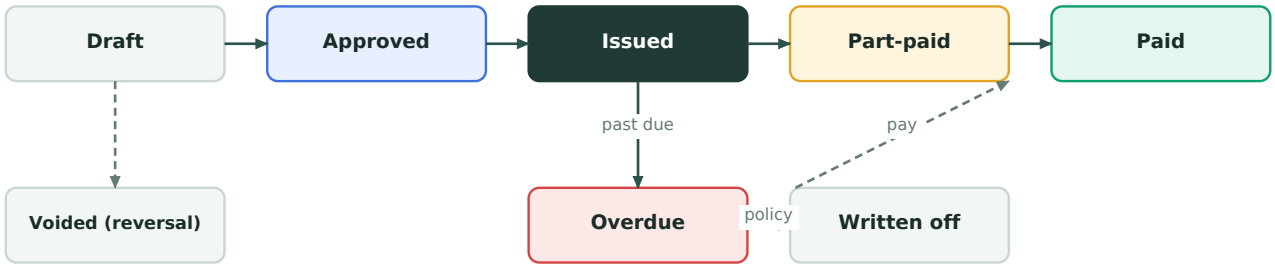


Fig 6.1 Bill lifecycle. Issued bills are never edited: corrections are adjustments or reversals.

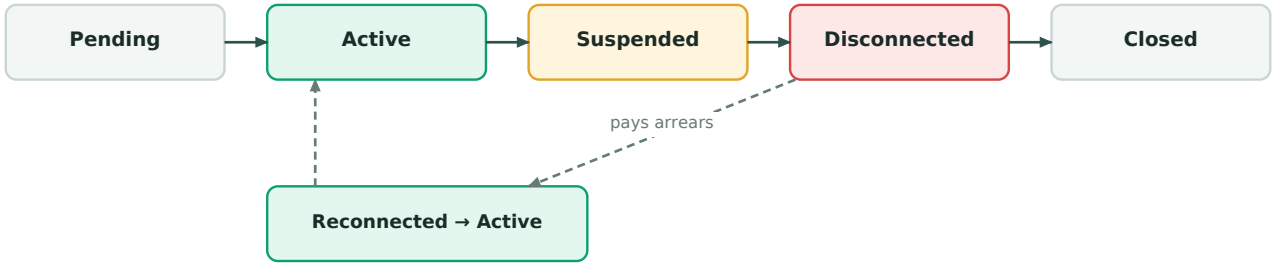


Fig 6.2 Account lifecycle. Each transition is audited and, where it affects revenue, needs approval.

// 7. Partitioning, indexing and retention

Table	Partition	Primary access path (index)	Retention (default, per tenant)
readings	monthly by read_at	(tenant_id, meter_install_id, read_at DESC)	7 years; photos 2 years then archived
bills, bill_lines	monthly by created_at	(tenant_id, account_id, created_at DESC); (tenant_id, status, due_date)	7 years
ledger_entries, journals	monthly by created_at	(tenant_id, account_id, created_at); (tenant_id, journal_id)	7 years minimum, never deleted in-period
payments, payment_inbox	monthly by received_at	UNIQUE (tenant_id, provider, provider_txn_id)	7 years; raw payload 2 years then compressed
audit_events	monthly by occurred_at	(tenant_id, entity_type, entity_id, occurred_at)	7 years; hash anchors kept indefinitely
notifications	monthly	(tenant_id, recipient, created_at)	12 months
outbox	daily, drop after publish + 7 days	(published_at) WHERE published_at IS NULL	short

Retention periods are defaults for planning. Actual periods must be confirmed with each tenant's legal and tax advisers (Kenyan record-keeping rules commonly require 5–7 years).

7.1 Data dictionary: critical enumerations

Field	Allowed values
bills.status	draft, approved, issued, partially_paid, paid, overdue, written_off, voided
accounts.status	pending, active, suspended, disconnected, closed
payments.method	mpesa_c2b, mpesa_stk, cash, bank, cheque, other
readings.source	mobile, web, import, estimate, smart_meter
readings.flag	none, high, low, zero, rollover, duplicate_suspect, photo_missing
ledger_accounts.type	asset, liability, revenue, expense, equity
billing_cycles.status	open, frozen, calculating, review, published, closed